

I'll build this as a practical interview-pattern template library: linear structures, linked lists, stacks/queues, trees, graphs, heaps, backtracking, DP, greedy, strings, bit/math, and design-style patterns. I'll keep the code templates clean and reusable rather than tying them to one specific LeetCode problem.

Below is a **complete practical coding-interview pattern library** in Python. No list can literally contain every possible algorithm ever invented, but this covers the major and minor reusable patterns that appear again and again in interviews: arrays, strings, linked lists, stacks, queues, heaps, trees, graphs, backtracking, dynamic programming, greedy, bit manipulation, math, and design-style questions.

Use this like a **template bank**: identify the pattern, copy the skeleton, then replace the marked condition/problem-specific logic.

0. Common Imports

```
PythonRunfromcollectionsimportdefaultdict, Counter, deque, OrderedDict
fromfunctoolsimportlru_cache
fromtypingimportList, Optional, Dict, Set, Tuple
importheapq
importmath
importbisect
```

1. Linear Data Structure Patterns

1. Single Pass Scan

Used for max/min, counting, running state, validation.

```
PythonRundefsingl_pass(nums: List[int]) -> int:
    answer=0
```

```
    forxinnums:
        # update answer/state
        answer+=x
```

```
    returnanswer
```

2. Frequency Map / Hash Map Counting

Used for anagrams, duplicates, majority, character counts.

```
PythonRundeffrequency_map(arr: List[int]) -> Dict[int, int]:  
    freq=defaultdict(int)
```

```
    forxinarr:  
        freq[x] +=1
```

```
    returnfreq
```

```
PythonRundefchar_frequency(s: str) -> Dict[str, int]:  
    returnCounter(s)
```

3. Hash Set Lookup

Used for duplicates, seen-before, two-sum style problems.

```
PythonRundefcontains_duplicate(nums: List[int]) -> bool:  
    seen=set()
```

```
    forxinnums:  
        ifxinseen:  
            returnTrue  
        seen.add(x)
```

```
    returnFalse
```

4. Two Sum Using Hash Map

```
PythonRundeftwo_sum(nums: List[int], target: int) -> List[int]:  
    seen= {}
```

```
    fori, xinenumerate(nums):  
        need=target-x
```

```
        ifneedinseen:  
            return [seen[need], i]
```

```
seen[x] = i
```

```
return []
```

5. Two Pointers — Opposite Direction

Used for sorted arrays, pair sum, palindrome, container area.

```
PythonRundeftwo_pointers_opposite(nums: List[int], target: int) -> bool:
```

```
left=0
```

```
right=len(nums) -1
```

```
whileleft<right:
```

```
total=nums[left] +nums[right]
```

```
iftotal==target:
```

```
returnTrue
```

```
eliftotal<target:
```

```
left+=1
```

```
else:
```

```
right-=1
```

```
returnFalse
```

6. Two Pointers — Same Direction

Used for removing duplicates, partitioning, slow-fast scanning.

```
PythonRundefremove_duplicates_sorted(nums: List[int]) -> int:
```

```
ifnotnums:
```

```
return0
```

```
slow=0
```

```
forfastinrange(1, len(nums)):
```

```
ifnums[fast] !=nums[slow]:
```

```
slow+=1
```

```
nums[slow] =nums[fast]
```

```
returnslow+1
```

7. Fast and Slow Pointers

Used for middle element, cycle detection, linked lists, arrays.

```
PythonRundeffast_slow_template(arr: List[int]) -> int:
```

```
slow=0
```

```
fast=0
```

```
whilefast<len(arr) andfast+1<len(arr):
```

```
slow+=1
```

```
fast+=2
```

```
returnslow
```

8. Sliding Window — Fixed Size

Used for max/min/sum of subarray of size k.

```
PythonRundeffixed_sliding_window(nums: List[int], k: int) -> int:
```

```
window_sum=0
```

```
best=float("-inf")
```

```
forrightinrange(len(nums)):
```

```
window_sum+=nums[right]
```

```
ifright>=k:
```

```
window_sum-=nums[right-k]
```

```
ifright>=k-1:
```

```
best=max(best, window_sum)
```

```
returnbest
```

9. Sliding Window — Variable Size, Longest Valid Window

Used for longest substring/subarray satisfying a condition.

PythonRundeflongest_valid_window(nums: List[int], limit: int) -> int:

left=0

current_sum=0

best=0

forrightinrange(len(nums)):

current_sum+=nums[right]

whilecurrent_sum>limit:

current_sum-=nums[left]

left+=1

best=max(best, right-left+1)

returnbest

10. Sliding Window — Minimum Valid Window

Used for minimum subarray length, minimum window substring.

PythonRundefmin_valid_window(nums: List[int], target: int) -> int:

left=0

current_sum=0

answer=float("inf")

forrightinrange(len(nums)):

current_sum+=nums[right]

whilecurrent_sum>=target:

answer=min(answer, right-left+1)

current_sum-=nums[left]

left+=1

return0ifanswer==float("inf") elseanswer

11. Sliding Window With Hash Map

Used for longest substring without repeating characters.

```
PythonRundeflongest_unique_substring(s: str) -> int:
```

```
    seen= {}
```

```
    left=0
```

```
    best=0
```

```
    for right, ch in enumerate(s):
```

```
        if ch in seen and seen[ch] >= left:
```

```
            left=seen[ch] + 1
```

```
        seen[ch] = right
```

```
        best=max(best, right-left+1)
```

```
    return best
```

12. At Most K Pattern

Used for “exactly K” problems.

```
PythonRundefat_most_k(nums: List[int], k: int) -> int:
```

```
    left=0
```

```
    count=0
```

```
    result=0
```

```
    for right in range(len(nums)):
```

```
        if nums[right] % 2 == 1:
```

```
            count+=1
```

```
    while count > k:
```

```
        if nums[left] % 2 == 1:
```

```
            count-=1
```

```
        left+=1
```

```
    result+=right-left+1
```

```
    return result
```

```
def exactly_k(nums: List[int], k: int) -> int:
    return at_most_k(nums, k) - at_most_k(nums, k-1)
```

13. Prefix Sum

Used for range sum queries.

```
PythonRun def build_prefix_sum(nums: List[int]) -> List[int]:
    prefix = [0]
```

```
    for i, num in enumerate(nums):
        prefix.append(prefix[-1] + num)
```

```
    return prefix
```

```
def range_sum(prefix: List[int], left: int, right: int) -> int:
    return prefix[right+1] - prefix[left]
```

14. Prefix Sum + Hash Map

Used for subarray sum equals target.

```
PythonRun def subarray_sum_equals_k(nums: List[int], k: int) -> int:
    count = 0
    prefix_sum = 0
    seen = defaultdict(int)
    seen[0] = 1
```

```
    for i, num in enumerate(nums):
        prefix_sum += num
```

```
        count += seen[prefix_sum - k]
```

```
        seen[prefix_sum] += 1
```

```
    return count
```

15. Difference Array

Used for range updates.

PythonRundefdifference_array(n: int, updates: List[Tuple[int, int, int]]) -> List[int]:

```
diff= [0] * (n+1)
```

```
forleft, right, valueinupdates:
```

```
diff[left] +=value
```

```
ifright+1<n:
```

```
diff[right+1] -=value
```

```
arr= [0] *n
```

```
running=0
```

```
foriinrange(n):
```

```
running+=diff[i]
```

```
arr[i] =running
```

```
returnarr
```

16. Prefix/Suffix Product

Used for product except self.

PythonRundefproduct_except_self(nums: List[int]) -> List[int]:

```
n=len(nums)
```

```
result= [1] *n
```

```
prefix=1
```

```
foriinrange(n):
```

```
result[i] =prefix
```

```
prefix*=nums[i]
```

```
suffix=1
```

```
foriinrange(n-1, -1, -1):
```

```
result[i] *=suffix
```

```
suffix*=nums[i]
```

```
returnresult
```

17. Kadane's Algorithm

Used for maximum subarray sum.

```
PythonRundefmax_subarray_sum(nums: List[int]) -> int:  
best=nums[0]  
current=nums[0]
```

```
forxinnums[1:]:  
    current=max(x, current+x)  
    best=max(best, current)
```

```
returnbest
```

18. Dutch National Flag / Three-Way Partition

Used for sorting 0s, 1s, 2s.

```
PythonRundefdutch_flag(nums: List[int]) -> None:  
low=0  
mid=0  
high=len(nums) -1
```

```
whilemid<=high:  
    ifnums[mid] ==0:  
        nums[low], nums[mid] =nums[mid], nums[low]  
        low+=1  
        mid+=1  
    elifnums[mid] ==1:  
        mid+=1  
    else:  
        nums[mid], nums[high] =nums[high], nums[mid]  
        high-=1
```

19. Binary Search — Standard

```
PythonRundefbinary_search(nums: List[int], target: int) -> int:  
left=0  
right=len(nums) -1
```

```
while left <= right:
    mid = left + (right - left) // 2

    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1

return -1
```

20. Binary Search — Lower Bound

First index where $\text{nums}[i] \geq \text{target}$.

```
PythonRun def lower_bound(nums: List[int], target: int) -> int:
    left = 0
    right = len(nums)

    while left < right:
        mid = left + (right - left) // 2

        if nums[mid] >= target:
            right = mid
        else:
            left = mid + 1

    return left
```

21. Binary Search — Upper Bound

First index where $\text{nums}[i] > \text{target}$.

```
PythonRun def upper_bound(nums: List[int], target: int) -> int:
    left = 0
    right = len(nums)

    while left < right:
```

```
mid=left+ (right-left) //2
```

```
if nums[mid] > target:
```

```
    right=mid
```

```
else:
```

```
    left=mid+1
```

```
return left
```

22. Binary Search on Answer

Used for minimum speed, capacity, days, maximum minimum value.

```
PythonRundefbinary_search_answer(left: int, right: int) -> int:
```

```
    def feasible(x: int) -> bool:
```

```
        # return True if x satisfies the problem condition
```

```
        return True
```

```
    while left < right:
```

```
        mid=left+ (right-left) //2
```

```
    if feasible(mid):
```

```
        right=mid
```

```
    else:
```

```
        left=mid+1
```

```
    return left
```

23. Rotated Sorted Array — Find Minimum

```
PythonRundef find_min_rotated(nums: List[int]) -> int:
```

```
    left=0
```

```
    right=len(nums) -1
```

```
    while left < right:
```

```
        mid=left+ (right-left) //2
```

```
    if nums[mid] > nums[right]:
```

```
        left=mid+1
```

```
    else:
```

```
right=mid
```

```
return nums[left]
```

24. Rotated Sorted Array — Search Target

```
PythonRundefsearch_rotated(nums: List[int], target: int) -> int:
```

```
left=0
```

```
right=len(nums) -1
```

```
while left<=right:
```

```
mid=left+ (right-left) //2
```

```
if nums[mid] ==target:
```

```
return mid
```

```
if nums[left] <=nums[mid]:
```

```
if nums[left] <=target<nums[mid]:
```

```
right=mid-1
```

```
else:
```

```
left=mid+1
```

```
else:
```

```
if nums[mid] <target<=nums[right]:
```

```
left=mid+1
```

```
else:
```

```
right=mid-1
```

```
return -1
```

2. Sorting, Intervals, and Sweep Line Patterns

25. Merge Intervals

```
PythonRundefmerge_intervals(intervals: List[List[int]]) -> List[List[int]]:
```

```
if not intervals:
```

```
return []
```

```
intervals.sort(key=lambdax: x[0])
```

```
merged= [intervals[0]]
```

```

for start, end in intervals[1:]:
    last_end = merged[-1][1]

    if start <= last_end:
        merged[-1][1] = max(last_end, end)
    else:
        merged.append([start, end])

return merged

```

26. Insert Interval

```

PythonRun def insert_interval(intervals: List[List[int]], new_interval: List[int]) -> List[List[int]]:
    result = []
    i = 0
    n = len(intervals)

    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
        i += 1

    while i < n and intervals[i][0] <= new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1

    result.append(new_interval)

    while i < n:
        result.append(intervals[i])
        i += 1

    return result

```

27. Meeting Rooms / Min Overlap Rooms

```

PythonRun def min_meeting_rooms(intervals: List[List[int]]) -> int:
    if not intervals:

```

```

return 0

starts=sorted(i[0] for i in intervals)
ends=sorted(i[1] for i in intervals)

s=0
e=0
rooms=0
max_rooms=0

while s<len(intervals):
    if starts[s] <ends[e]:
        rooms+=1
        max_rooms=max(max_rooms, rooms)
        s+=1
    else:
        rooms-=1
        e+=1

return max_rooms

```

28. Sweep Line

Used for range overlap, events, bookings, population count.

Python Run def sweep_line(intervals: List[List[int]]) -> int:
 events= []

```

for start, end in intervals:
    events.append((start, 1))
    events.append((end, -1))

```

```

events.sort()

```

```

current=0
best=0

```

```

for _, delta in events:
    current+=delta
    best=max(best, current)

```

returnbest

29. Quickselect

Used for kth largest/smallest.

```
PythonRundefkth_largest(nums: List[int], k: int) -> int:  
    target=len(nums) -k
```

```
    defpartition(left: int, right: int) -> int:  
        pivot=nums[right]  
        p=left
```

```
        foriinrange(left, right):  
            ifnums[i] <=pivot:  
                nums[p], nums[i] =nums[i], nums[p]  
                p+=1
```

```
        nums[p], nums[right] =nums[right], nums[p]  
        returnp
```

```
    left=0  
    right=len(nums) -1
```

```
    whileleft<=right:  
        pivot_index=partition(left, right)
```

```
        ifpivot_index==target:  
            returnnums[pivot_index]  
        elifpivot_index<target:  
            left=pivot_index+1  
        else:  
            right=pivot_index-1
```

```
    return-1
```

3. Stack and Queue Patterns

30. Basic Stack

PythonRundefstack_template(arr: List[int]) -> List[int]:

```
stack= []
```

```
forxinarr:
```

```
stack.append(x)
```

```
whilestackandFalse:
```

```
stack.pop()
```

```
returnstack
```

31. Valid Parentheses

PythonRundefvalid_parentheses(s: str) -> bool:

```
stack= []
```

```
pairs= {
```

```
    ")": "(",
```

```
    "]": "[",
```

```
    "}": "{"
```

```
    }
```

```
forchins:
```

```
    ifchinpairs.values():
```

```
        stack.append(ch)
```

```
    elifchinpairs:
```

```
        ifnotstackorstack[-1] !=pairs[ch]:
```

```
            returnFalse
```

```
        stack.pop()
```

```
returnnotstack
```

32. Monotonic Increasing Stack

Used for next smaller element.

```
PythonRundefnext_smaller(nums: List[int]) -> List[int]:
n=len(nums)
result= [-1] *n
stack= []
```

```
fori, xinenumerate(nums):
whilestackandnums[stack[-1]] >x:
index=stack.pop()
result[index] =x
```

```
stack.append(i)
```

```
returnresult
```

33. Monotonic Decreasing Stack

Used for next greater element.

```
PythonRundefnext_greater(nums: List[int]) -> List[int]:
n=len(nums)
result= [-1] *n
stack= []
```

```
fori, xinenumerate(nums):
whilestackandnums[stack[-1]] <x:
index=stack.pop()
result[index] =x
```

```
stack.append(i)
```

```
returnresult
```

34. Largest Rectangle in Histogram

```
PythonRundeflargest_rectangle_area(heights: List[int]) -> int:
stack= []
best=0
heights.append(0)
```

```
fori, hinenumerate(heights):
```

```
while stack and heights[stack[-1]] > h:
    height = heights[stack.pop()]
    left_boundary = stack[-1] if stack else -1
    width = i - left_boundary - 1
    best = max(best, height * width)
```

```
stack.append(i)
```

```
heights.pop()
return best
```

35. Monotonic Queue

Used for sliding window maximum.

```
PythonRun def max_sliding_window(nums: List[int], k: int) -> List[int]:
    dq = deque()
    result = []
```

```
    for i, x in enumerate(nums):
        while dq and dq[0] <= i - k:
            dq.popleft()
```

```
        while dq and nums[dq[-1]] <= x:
            dq.pop()
```

```
        dq.append(i)
```

```
    if i >= k - 1:
        result.append(nums[dq[0]])
```

```
    return result
```

4. Linked List Patterns

36. Linked List Node

```
PythonRun class ListNode:
    def __init__(self, val: int = 0, next: Optional["ListNode"] = None):
```

```
self.val =val  
self.next =next
```

37. Reverse Linked List

```
PythonRundefreverse_list(head: Optional[ListNode]) -> Optional[ListNode]:  
prev=None  
current=head
```

```
whilecurrent:  
nxt=current.next  
current.next =prev  
prev=current  
current=nxt
```

```
returnprev
```

38. Find Middle of Linked List

```
PythonRundefmiddle_node(head: Optional[ListNode]) -> Optional[ListNode]:  
slow=head  
fast=head
```

```
whilefastandfast.next:  
slow=slow.next  
fast=fast.next.next
```

```
returnslow
```

39. Detect Cycle in Linked List

```
PythonRundefhas_cycle(head: Optional[ListNode]) -> bool:  
slow=head  
fast=head
```

```
whilefastandfast.next:  
slow=slow.next  
fast=fast.next.next
```

```
ifslow==fast:  
    returnTrue
```

```
returnFalse
```

40. Find Cycle Start

PythonRundefdetect_cycle_start(head: Optional[ListNode]) -> Optional[ListNode]:

```
slow=head
```

```
fast=head
```

```
whilefastandfast.next:
```

```
    slow=slow.next
```

```
    fast=fast.next.next
```

```
ifslow==fast:
```

```
    break
```

```
else:
```

```
    returnNone
```

```
slow=head
```

```
whileslow!=fast:
```

```
    slow=slow.next
```

```
    fast=fast.next
```

```
returnslow
```

41. Merge Two Sorted Lists

PythonRundefmerge_two_lists(l1: Optional[ListNode], l2: Optional[ListNode]) ->

Optional[ListNode]:

```
dummy=ListNode()
```

```
current=dummy
```

```
whilel1andl2:
```

```
    ifl1.val <=l2.val:
```

```
        current.next =l1
```

```
        l1=l1.next
```

```
    else:
```

```
current.next = l2
l2 = l2.next

current = current.next

current.next = l1 if l1 else l2

return dummy.next
```

42. Remove Nth Node From End

```
PythonRun def remove_nth_from_end(head: Optional[ListNode], n: int) -> Optional[ListNode]:
    dummy = ListNode(0, head)
    slow = dummy
    fast = dummy

    for _ in range(n):
        fast = fast.next

    while fast.next:
        slow = slow.next
        fast = fast.next

    slow.next = slow.next.next

    return dummy.next
```

43. Reverse Nodes in K Group

```
PythonRun def reverse_k_group(head: Optional[ListNode], k: int) -> Optional[ListNode]:
    def get_kth(node: ListNode, k: int) -> Optional[ListNode]:
        while node and k > 0:
            node = node.next
            k -= 1
        return node

    dummy = ListNode(0, head)
    group_prev = dummy
```

```
while True:
    kth = get_kth(group_prev, k)
```

```
    if not kth:
        break
```

```
    group_next = kth.next
```

```
    prev = group_next
    current = group_prev.next
```

```
    while current != group_next:
        tmp = current.next
        current.next = prev
        prev = current
        current = tmp
```

```
    tmp = group_prev.next
    group_prev.next = kth
    group_prev = tmp
```

```
    return dummy.next
```

5. Tree Patterns

44. Binary Tree Node

```
PythonRunclassTreeNode:
    def __init__(
        self,
        val: int = 0,
        left: Optional["TreeNode"] = None,
        right: Optional["TreeNode"] = None
    ):
        self.val = val
        self.left = left
        self.right = right
```

45. Recursive DFS Traversal

PythonRun

```
def preorder(root: Optional[TreeNode]) -> List[int]:
```

```
    result = []
```

```
    def dfs(node: Optional[TreeNode]) -> None:
```

```
        if not node:
```

```
            return
```

```
        result.append(node.val)
```

```
        dfs(node.left)
```

```
        dfs(node.right)
```

```
    dfs(root)
```

```
    return result
```

PythonRun

```
def inorder(root: Optional[TreeNode]) -> List[int]:
```

```
    result = []
```

```
    def dfs(node: Optional[TreeNode]) -> None:
```

```
        if not node:
```

```
            return
```

```
        dfs(node.left)
```

```
        result.append(node.val)
```

```
        dfs(node.right)
```

```
    dfs(root)
```

```
    return result
```

PythonRun

```
def postorder(root: Optional[TreeNode]) -> List[int]:
```

```
    result = []
```

```
    def dfs(node: Optional[TreeNode]) -> None:
```

```
        if not node:
```

```
            return
```

```
        dfs(node.left)
```

```
        dfs(node.right)
```

```
        result.append(node.val)
```

```
    dfs(root)
```

```
    return result
```

46. Iterative DFS

```
PythonRundefiterative_preorder(root: Optional[TreeNode]) -> List[int]:
if not root:
    return []

stack = [root]
result = []

while stack:
    node = stack.pop()
    result.append(node.val)

    if node.right:
        stack.append(node.right)
    if node.left:
        stack.append(node.left)

return result
```

47. BFS Level Order Traversal

```
PythonRundeflevel_order(root: Optional[TreeNode]) -> List[List[int]]:
if not root:
    return []

result = []
queue = deque([root])

while queue:
    level = []

    for _ in range(len(queue)):
        node = queue.popleft()
        level.append(node.val)

        if node.left:
            queue.append(node.left)
        if node.right:
```

```
queue.append(node.right)
```

```
result.append(level)
```

```
return result
```

48. Tree Height / Depth

```
PythonRun def max_depth(root: Optional[TreeNode]) -> int:
```

```
    if not root:
```

```
        return 0
```

```
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

49. Path Sum

```
PythonRun def has_path_sum(root: Optional[TreeNode], target_sum: int) -> bool:
```

```
    if not root:
```

```
        return False
```

```
    if not root.left and not root.right:
```

```
        return root.val == target_sum
```

```
    remaining = target_sum - root.val
```

```
    return (
```

```
        has_path_sum(root.left, remaining) or
```

```
        has_path_sum(root.right, remaining)
```

```
    )
```

50. Lowest Common Ancestor — Binary Tree

```
PythonRun def lowest_common_ancestor(
```

```
    root: Optional[TreeNode],
```

```
    p: TreeNode,
```

```
    q: TreeNode
```

```
) -> Optional[TreeNode]:
```

```
    if not root or root == p or root == q:
```

```
        return root
```

```
left=lowest_common_ancestor(root.left, p, q)
right=lowest_common_ancestor(root.right, p, q)
```

```
if left and right:
    return root
```

```
return left if left else right
```

51. Validate BST

```
PythonRun def is_valid_bst(root: Optional[TreeNode]) -> bool:
    def dfs(node: Optional[TreeNode], low: float, high: float) -> bool:
```

```
        if not node:
            return True
```

```
        if not (low < node.val < high):
            return False
```

```
        return (
            dfs(node.left, low, node.val) and
            dfs(node.right, node.val, high)
        )
```

```
    return dfs(root, float("-inf"), float("inf"))
```

52. BST Search

```
PythonRun def search_bst(root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
```

```
    while root:
        if root.val == val:
            return root
        elif val < root.val:
            root = root.left
        else:
            root = root.right
```

```
    return None
```

53. Kth Smallest in BST

PythonRundefkth_smallest(root: Optional[TreeNode], k: int) -> int:

stack= []

current=root

whilestackorcurrent:

whilecurrent:

stack.append(current)

current=current.left

current=stack.pop()

k-=1

ifk==0:

returncurrent.val

current=current.right

return-1

54. Serialize and Deserialize Binary Tree

PythonRundefserialize(root: Optional[TreeNode]) -> str:

values= []

defdfs(node: Optional[TreeNode]) -> None:

ifnotnode:

values.append("#")

return

values.append(str(node.val))

dfs(node.left)

dfs(node.right)

dfs(root)

return",".join(values)

defdeserialize(data: str) -> Optional[TreeNode]:

values=iter(data.split(","))

```
defdfs() -> Optional[TreeNode]:  
    val=next(values)
```

```
    ifval=="#":  
        returnNone
```

```
    node=TreeNode(int(val))  
    node.left =dfs()  
    node.right =dfs()
```

```
    returnnode
```

```
    returndfs()
```

6. Trie Pattern

55. Trie / Prefix Tree

```
PythonRunclassTrieNode:  
    def__init__(self):  
        self.children = {}  
        self.is_word =False
```

```
classTrie:  
    def__init__(self):  
        self.root =TrieNode()
```

```
    definsert(self, word: str) -> None:  
        node=self.root
```

```
        forchinword:  
            ifchnotinnode.children:  
                node.children[ch] =TrieNode()
```

```
        node=node.children[ch]
```

```
        node.is_word =True
```

```

def search(self, word: str) -> bool:
    node = self.root

    for ch in word:
        if ch not in node.children:
            return False

        node = node.children[ch]

    return node.is_word

def starts_with(self, prefix: str) -> bool:
    node = self.root

    for ch in prefix:
        if ch not in node.children:
            return False

        node = node.children[ch]

    return True

```

7. Graph Patterns

56. Build Adjacency List

```

PythonRun def build_graph(n: int, edges: List[List[int]], directed: bool = False) -> Dict[int, List[int]]:
    graph = defaultdict(list)

    for u, v in edges:
        graph[u].append(v)

        if not directed:
            graph[v].append(u)

    return graph

```

57. Graph DFS — Recursive

```
PythonRun
def graph_dfs(n: int, edges: List[List[int]], start: int) -> List[int]:
    graph = build_graph(n, edges)
    visited = set()
    result = []

    def dfs(node: int) -> None:
        if node in visited:
            return

        visited.add(node)
        result.append(node)

        for nei in graph[node]:
            dfs(nei)

    dfs(start)
    return result
```

58. Graph DFS — Iterative

```
PythonRun
def graph_dfs_iterative(n: int, edges: List[List[int]], start: int) -> List[int]:
    graph = build_graph(n, edges)
    visited = set()
    stack = [start]
    result = []

    while stack:
        node = stack.pop()

        if node in visited:
            continue

        visited.add(node)
        result.append(node)

        for nei in graph[node]:
            if nei not in visited:
                stack.append(nei)
```

```
returnresult
```

59. Graph BFS

```
PythonRundefgraph_bfs(n: int, edges: List[List[int]], start: int) -> List[int]:
graph=build_graph(n, edges)
visited= {start}
queue=deque([start])
result= []
```

```
whilequeue:
node=queue.popleft()
result.append(node)
```

```
forneiingraph[node]:
ifneinotinvisited:
visited.add(nei)
queue.append(nei)
```

```
returnresult
```

60. BFS Shortest Path — Unweighted Graph

```
PythonRundefshortest_path_unweighted(n: int, edges: List[List[int]], start: int) ->
List[int]:
graph=build_graph(n, edges)
dist= [-1] *n
dist[start] =0
```

```
queue=deque([start])
```

```
whilequeue:
node=queue.popleft()
```

```
forneiingraph[node]:
ifdist[nei] ==-1:
dist[nei] =dist[node] +1
queue.append(nei)
```

returndist

61. Grid DFS

PythonRundefgrid_dfs(grid: List[List[str]]) -> int:

rows=len(grid)

cols=len(grid[0])

visited=set()

defdfs(r: int, c: int) -> None:

if (

r<0orr>=rowsor

c<0orc>=colsor

(r, c) invisitedor

grid[r][c] == "0"

):

return

visited.add((r, c))

fordr, dcin [(1, 0), (-1, 0), (0, 1), (0, -1)]:

dfs(r+dr, c+dc)

count=0

forrinrange(rows):

forcinrange(cols):

ifgrid[r][c] == "1"and (r, c) notinvisited:

dfs(r, c)

count+=1

returncount

62. Grid BFS

PythonRundefgrid_bfs(grid: List[List[str]], start_r: int, start_c: int) -> None:

rows=len(grid)

cols=len(grid[0])

```

queue=deque([(start_r, start_c)])
visited= {(start_r, start_c)}

while queue:
    r, c=queue.popleft()

    for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
        nr=r+dr
        nc=c+dc

        if (
            0<=nr<rows and
            0<=nc<cols and
            (nr, nc) not in visited and
            grid[nr][nc] != "0"
        ):
            visited.add((nr, nc))
            queue.append((nr, nc))

```

63. Topological Sort — Kahn's Algorithm

```

Python
def topological_sort_kahn(n: int, edges: List[List[int]]) -> List[int]:
    graph=defaultdict(list)
    indegree= [0] *n

    for u, v in edges:
        graph[u].append(v)
        indegree[v] +=1

    queue=deque([i for i in range(n) if indegree[i] ==0])
    order= []

    while queue:
        node=queue.popleft()
        order.append(node)

        for nei in graph[node]:
            indegree[nei] -=1

            if indegree[nei] ==0:
                queue.append(nei)

```

```
return order if len(order) == n else []
```

64. Topological Sort — DFS

```
PythonRundeftopological_sort_dfs(n: int, edges: List[List[int]]) -> List[int]:  
graph=defaultdict(list)
```

```
for u, v in edges:  
graph[u].append(v)
```

```
visited= [0] *n  
order= []  
has_cycle=False
```

```
def dfs(node: int) -> None:  
nonlocal has_cycle
```

```
if visited[node] ==1:  
has_cycle=True  
return
```

```
if visited[node] ==2:  
return
```

```
visited[node] =1
```

```
for nei in graph[node]:  
dfs(nei)
```

```
visited[node] =2  
order.append(node)
```

```
for i in range(n):  
if visited[i] ==0:  
dfs(i)
```

```
if has_cycle:  
return []
```

```
return order[::-1]
```

65. Union Find / Disjoint Set Union

```
PythonRunclass UnionFind:
```

```
def __init__(self, n: int):
```

```
    self.parent = list(range(n))
```

```
    self.rank = [0] * n
```

```
    self.count = n
```

```
def find(self, x: int) -> int:
```

```
    if self.parent[x] != x:
```

```
        self.parent[x] = self.find(self.parent[x])
```

```
    return self.parent[x]
```

```
def union(self, x: int, y: int) -> bool:
```

```
    root_x = self.find(x)
```

```
    root_y = self.find(y)
```

```
    if root_x == root_y:
```

```
        return False
```

```
    if self.rank[root_x] < self.rank[root_y]:
```

```
        self.parent[root_x] = root_y
```

```
    elif self.rank[root_x] > self.rank[root_y]:
```

```
        self.parent[root_y] = root_x
```

```
    else:
```

```
        self.parent[root_y] = root_x
```

```
        self.rank[root_x] += 1
```

```
    self.count -= 1
```

```
    return True
```

66. Detect Cycle — Undirected Graph

```
PythonRun def has_cycle_undirected(n: int, edges: List[List[int]]) -> bool:
```

```
    uf = UnionFind(n)
```

```
for u, v in edges:
    if not u in graph:
        graph[u] = []
    if not v in graph:
        graph[v] = []
    graph[u].append(v)
```

```
return False
```

67. Detect Cycle — Directed Graph

Python Run def has_cycle_directed(n: int, edges: List[List[int]]) -> bool:
graph = defaultdict(list)

```
for u, v in edges:
    graph[u].append(v)
```

```
state = [0] * n
```

```
def dfs(node: int) -> bool:
    if state[node] == 1:
        return True
```

```
    if state[node] == 2:
        return False
```

```
    state[node] = 1
```

```
    for nei in graph[node]:
        if dfs(nei):
            return True
```

```
    state[node] = 2
    return False
```

```
for i in range(n):
    if state[i] == 0 and dfs(i):
        return True
```

```
return False
```

68. Bipartite Graph Check

PythonRundefis_bipartite(graph: List[List[int]]) -> bool:

n=len(graph)

color= {}

forstartinrange(n):

ifstartincolor:

continue

queue=deque([start])

color[start] =0

whilequeue:

node=queue.popleft()

forneiingraph[node]:

ifneinotincolor:

color[nei] =1-color[node]

queue.append(nei)

elifcolor[nei] ==color[node]:

returnFalse

returnTrue

69. Dijkstra's Algorithm

Used for shortest path with non-negative weights.

PythonRundefdijkstra(n: int, edges: List[List[int]], start: int) -> List[float]:

graph=defaultdict(list)

foru, v, winedges:

graph[u].append((v, w))

dist= [float("inf")] *n

dist[start] =0

heap= [(0, start)]

whileheap:

```
current_dist, node=heapq.heappop(heap)
```

```
if current_dist > dist[node]:  
    continue
```

```
for nei, weight in graph[node]:  
    new_dist = current_dist + weight
```

```
if new_dist < dist[nei]:  
    dist[nei] = new_dist  
    heapq.heappush(heap, (new_dist, nei))
```

```
return dist
```

70. Bellman-Ford Algorithm

Used when negative weights may exist.

```
PythonRun def bellman_ford(n: int, edges: List[List[int]], start: int) -> Optional[List[float]]:
```

```
    dist = [float("inf")] * n  
    dist[start] = 0
```

```
    for _ in range(n-1):  
        changed = False
```

```
        for u, v, w in edges:  
            if dist[u] != float("inf") and dist[u] + w < dist[v]:  
                dist[v] = dist[u] + w  
                changed = True
```

```
        if not changed:  
            break
```

```
        for u, v, w in edges:  
            if dist[u] != float("inf") and dist[u] + w < dist[v]:  
                return None
```

```
    return dist
```

71. Floyd-Warshall Algorithm

All-pairs shortest path.

```
PythonRundeffloyd_warshall(n: int, edges: List[List[int]]) -> List[List[float]]:
```

```
dist= [[float("inf")] *nfor_inrange(n)]
```

```
foriinrange(n):
```

```
dist[i][i] =0
```

```
foru, v, winedges:
```

```
dist[u][v] =min(dist[u][v], w)
```

```
forkinrange(n):
```

```
foriinrange(n):
```

```
forjrange(n):
```

```
ifdist[i][k] +dist[k][j] <dist[i][j]:
```

```
dist[i][j] =dist[i][k] +dist[k][j]
```

```
returndist
```

72. Minimum Spanning Tree — Kruskal

```
PythonRundefkruskal(n: int, edges: List[List[int]]) -> int:
```

```
edges.sort(key=lambdax: x[2])
```

```
uf=UnionFind(n)
```

```
total=0
```

```
foru, v, winedges:
```

```
ifuf.union(u, v):
```

```
total+=w
```

```
returntotal
```

73. Minimum Spanning Tree — Prim

```
PythonRundefprim(n: int, edges: List[List[int]]) -> int:
```

```
graph=defaultdict(list)
```

```
foru, v, winedges:
```

```

graph[u].append((w, v))
graph[v].append((w, u))

visited=set()
heap= [(0, 0)]
total=0

whileheapandlen(visited) <n:
weight, node=heapq.heappop(heap)

ifnodeinvisited:
continue

visited.add(node)
total+=weight

fornext_weight, neiingraph[node]:
ifneinotinvisited:
heapq.heappush(heap, (next_weight, nei))

returntotaliflen(visited) ==nelse-1

```

8. Backtracking Patterns

74. General Backtracking Template

```

PythonRundefbacktrack_template(nums: List[int]) -> List[List[int]]:
result= []
path= []

defbacktrack(index: int) -> None:
ifindex==len(nums):
result.append(path[:])
return

# choice 1: include
path.append(nums[index])
backtrack(index+1)
path.pop()

```

```
# choice 2: exclude  
backtrack(index+1)
```

```
backtrack(0)  
return result
```

75. Subsets

```
PythonRun def subsets(nums: List[int]) -> List[List[int]]:  
    result = []  
    path = []
```

```
    def backtrack(start: int) -> None:  
        result.append(path[:])
```

```
        for i in range(start, len(nums)):  
            path.append(nums[i])  
            backtrack(i+1)  
            path.pop()
```

```
    backtrack(0)  
    return result
```

76. Subsets With Duplicates

```
PythonRun def subsets_with_dup(nums: List[int]) -> List[List[int]]:  
    nums.sort()  
    result = []  
    path = []
```

```
    def backtrack(start: int) -> None:  
        result.append(path[:])
```

```
        for i in range(start, len(nums)):  
            if i > start and nums[i] == nums[i-1]:  
                continue
```

```
            path.append(nums[i])  
            backtrack(i+1)  
            path.pop()
```

```
backtrack(0)
return result
```

77. Combinations

```
PythonRun def combine(n: int, k: int) -> List[List[int]]:
    result = []
    path = []
```

```
    def backtrack(start: int) -> None:
        if len(path) == k:
            result.append(path[:])
            return
```

```
        for i in range(start, n+1):
            path.append(i)
            backtrack(i+1)
            path.pop()
```

```
    backtrack(1)
    return result
```

78. Permutations

```
PythonRun def permute(nums: List[int]) -> List[List[int]]:
    result = []
    path = []
    used = [False] * len(nums)
```

```
    def backtrack() -> None:
        if len(path) == len(nums):
            result.append(path[:])
            return
```

```
        for i in range(len(nums)):
            if used[i]:
                continue
```

```
            used[i] = True
```

```
path.append(nums[i])
```

```
backtrack()
```

```
path.pop()
used[i] = False
```

```
backtrack()
return result
```

79. Combination Sum

```
PythonRundefcombination_sum(candidates: List[int], target: int) -> List[List[int]]:
```

```
result= []
```

```
path= []
```

```
defbacktrack(start: int, remaining: int) -> None:
```

```
ifremaining==0:
```

```
result.append(path[:])
```

```
return
```

```
ifremaining<0:
```

```
return
```

```
foriinrange(start, len(candidates)):
```

```
path.append(candidates[i])
```

```
backtrack(i, remaining-candidates[i])
```

```
path.pop()
```

```
backtrack(0, target)
```

```
return result
```

80. Letter Combinations of Phone Number

```
PythonRundefletter_combinations(digits: str) -> List[str]:
```

```
ifnotdigits:
```

```
return []
```

```
mapping= {
```

```
"2": "abc",
```

```
"3": "def",
"4": "ghi",
"5": "jkl",
"6": "mno",
"7": "pqrs",
"8": "tuv",
"9": "wxyz"
}
```

```
result= []
path= []
```

```
def backtrack(index: int) -> None:
    if index==len(digits):
        result.append("".join(path))
        return
```

```
    for ch in mapping[digits[index]]:
        path.append(ch)
        backtrack(index+1)
    path.pop()
```

```
backtrack(0)
return result
```

81. Palindrome Partitioning

```
PythonRun def palindrome_partition(s: str) -> List[List[str]]:
    result= []
    path= []
```

```
    def is_palindrome(left: int, right: int) -> bool:
        while left<right:
            if s[left] != s[right]:
                return False
```

```
        left+=1
        right-=1
```

```
    return True
```

```

def backtrack(start: int) -> None:
    if start == len(s):
        result.append(path[:])
    return

```

```

    for end in range(start, len(s)):
        if is_palindrome(start, end):
            path.append(s[start:end+1])
            backtrack(end+1)
            path.pop()

```

```

    backtrack(0)
    return result

```

82. Word Search

```

PythonRun def exist(board: List[List[str]], word: str) -> bool:
    rows = len(board)
    cols = len(board[0])

```

```

    def backtrack(r: int, c: int, index: int) -> bool:
        if index == len(word):
            return True

```

```

        if (
            r < 0 or r >= rows or
            c < 0 or c >= cols or
            board[r][c] != word[index]
        ):
            return False

```

```

        temp = board[r][c]
        board[r][c] = "#"

```

```

        found = (
            backtrack(r+1, c, index+1) or
            backtrack(r-1, c, index+1) or
            backtrack(r, c+1, index+1) or
            backtrack(r, c-1, index+1)
        )

```

```
board[r][c] = temp
```

```
return found
```

```
for r in range(rows):
```

```
    for c in range(cols):
```

```
        if backtrack(r, c, 0):
```

```
            return True
```

```
return False
```

83. N-Queens

```
Python: def solve_n_queens(n: int) -> List[List[str]]:
```

```
    result = []
```

```
    board = [ "." * n for _ in range(n) ]
```

```
    cols = set()
```

```
    pos_diag = set()
```

```
    neg_diag = set()
```

```
    def backtrack(row: int) -> None:
```

```
        if row == n:
```

```
            result.append(["".join(r) for r in board])
```

```
            return
```

```
        for col in range(n):
```

```
            if col in cols or (row + col) in pos_diag or (row - col) in neg_diag:
```

```
                continue
```

```
        cols.add(col)
```

```
        pos_diag.add(row + col)
```

```
        neg_diag.add(row - col)
```

```
        board[row][col] = "Q"
```

```
        backtrack(row + 1)
```

```
    board[row][col] = "."
```

```
    cols.remove(col)
```

```
    pos_diag.remove(row + col)
```

```
    neg_diag.remove(row - col)
```

```
backtrack(0)
return result
```

9. Heap / Priority Queue Patterns

84. Min Heap

```
PythonRun def min_heap_template(nums: List[int]) -> List[int]:
    heap = []
```

```
    for x in nums:
        heapq.heappush(heap, x)
```

```
    result = []
```

```
    while heap:
        result.append(heapq.heappop(heap))
```

```
    return result
```

85. Max Heap

Python has no direct max heap, so use negative values.

```
PythonRun def max_heap_template(nums: List[int]) -> List[int]:
    heap = []
```

```
    for x in nums:
        heapq.heappush(heap, -x)
```

```
    result = []
```

```
    while heap:
        result.append(-heapq.heappop(heap))
```

```
    return result
```

86. Top K Largest

```
PythonRundeftop_k_largest(nums: List[int], k: int) -> List[int]:  
    heap= []
```

```
    forxinnums:  
        heapq.heappush(heap, x)
```

```
    iflen(heap) >k:  
        heapq.heappop(heap)
```

```
    returnheap
```

87. K Smallest Values

```
PythonRundefk_smallest(nums: List[int], k: int) -> List[int]:  
    heap= []
```

```
    forxinnums:  
        heapq.heappush(heap, -x)
```

```
    iflen(heap) >k:  
        heapq.heappop(heap)
```

```
    return [-xforxinheap]
```

88. K-Way Merge

```
PythonRundefmerge_k_sorted_lists(lists: List[List[int]]) -> List[int]:  
    heap= []
```

```
    forlist_index, arrinenumerate(lists):  
        ifarr:  
            heapq.heappush(heap, (arr[0], list_index, 0))
```

```
    result= []
```

```
    whileheap:  
        value, list_index, element_index=heapq.heappop(heap)  
        result.append(value)
```

```
next_index=element_index+1

if next_index<len(lists[list_index]):
    next_value=list(list_index)[next_index]
    heapq.heappush(heap, (next_value, list_index, next_index))

return result
```

89. Two Heaps — Median Finder

PythonRunclassMedianFinder:

```
def __init__(self):
```

```
    self.small = []
```

```
    self.large = []
```

```
def add_num(self, num: int) -> None:
```

```
    heapq.heappush(self.small, -num)
```

```
    if self.small and self.large and -self.small[0] > self.large[0]:
```

```
        value = -heapq.heappop(self.small)
```

```
        heapq.heappush(self.large, value)
```

```
    if len(self.small) > len(self.large) + 1:
```

```
        value = -heapq.heappop(self.small)
```

```
        heapq.heappush(self.large, value)
```

```
    if len(self.large) > len(self.small):
```

```
        value = heapq.heappop(self.large)
```

```
        heapq.heappush(self.small, -value)
```

```
def find_median(self) -> float:
```

```
    if len(self.small) > len(self.large):
```

```
        return -self.small[0]
```

```
    return (-self.small[0] + self.large[0]) / 2
```

90. Heap With Lazy Deletion

```
PythonRunclassLazyHeap:
def __init__(self):
self.heap = []
self.deleted = Counter()

def push(self, x: int) -> None:
heapq.heappush(self.heap, x)

def delete(self, x: int) -> None:
self.deleted[x] += 1

def clean(self) -> None:
while self.heap and self.deleted[self.heap[0]] > 0:
value = heapq.heappop(self.heap)
self.deleted[value] -= 1

def top(self) -> int:
self.clean()
return self.heap[0]

def pop(self) -> int:
self.clean()
return heapq.heappop(self.heap)
```

10. Dynamic Programming Patterns

91. Top-Down DP With Memoization

```
PythonRundef top_down_dp(n: int) -> int:
@lru_cache(None)
def dp(i: int) -> int:
if i <= 1:
return i

return dp(i-1) + dp(i-2)

return dp(n)
```

92. Bottom-Up DP

```
PythonRun
def bottom_up_dp(n: int) -> int:
    if n <= 1:
        return n

    dp = [0] * (n+1)
    dp[1] = 1

    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]

    return dp[n]
```

93. Space-Optimized DP

```
PythonRun
def space_optimized_dp(n: int) -> int:
    if n <= 1:
        return n

    prev2 = 0
    prev1 = 1

    for _ in range(2, n+1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

    return prev1
```

94. 1D DP — Climbing Stairs Pattern

```
PythonRun
def climb_stairs(n: int) -> int:
    dp = [0] * (n+1)
    dp[0] = 1

    for i in range(1, n+1):
        dp[i] += dp[i-1]

    if i >= 2:
```

```
dp[i] += dp[i-2]
```

```
returndp[n]
```

95. 2D Grid DP

PythonRundefunique_paths(m: int, n: int) -> int:

```
dp= [[1] *nfor_inrange(m)]
```

```
forrinrange(1, m):
```

```
forcinrange(1, n):
```

```
dp[r][c] =dp[r-1][c] +dp[r][c-1]
```

```
returndp[m-1][n-1]
```

96. 0/1 Knapsack

PythonRundefzero_one_knapsack(weights: List[int], values: List[int], capacity: int) -> int:

```
n=len(weights)
```

```
dp= [[0] * (capacity+1) for_inrange(n+1)]
```

```
foriinrange(1, n+1):
```

```
weight=weights[i-1]
```

```
value=values[i-1]
```

```
forcapinrange(capacity+1):
```

```
dp[i][cap] =dp[i-1][cap]
```

```
ifcap>=weight:
```

```
dp[i][cap] =max(
```

```
dp[i][cap],
```

```
value+dp[i-1][cap-weight]
```

```
)
```

```
returndp[n][capacity]
```

97. 0/1 Knapsack — Space Optimized

```
PythonRundefzero_one_knapsack_optimized(weights: List[int], values: List[int], capacity:
int) -> int:
    dp= [0] * (capacity+1)

    forweight, valueinzip(weights, values):
        forcapinrange(capacity, weight-1, -1):
            dp[cap] =max(dp[cap], value+dp[cap-weight])

    returndp[capacity]
```

98. Unbounded Knapsack

```
PythonRundefunbounded_knapsack(weights: List[int], values: List[int], capacity: int) ->
int:
    dp= [0] * (capacity+1)

    forcapinrange(capacity+1):
        forweight, valueinzip(weights, values):
            ifcap>=weight:
                dp[cap] =max(dp[cap], value+dp[cap-weight])

    returndp[capacity]
```

99. Coin Change — Minimum Coins

```
PythonRundefcoin_change_min(coins: List[int], amount: int) -> int:
    dp= [float("inf")] * (amount+1)
    dp[0] =0

    fortotalinrange(1, amount+1):
        forcoinincoins:
            iftotal>=coin:
                dp[total] =min(dp[total], 1+dp[total-coin])

    return-1ifdp[amount] ==float("inf") elsedp[amount]
```

100. Coin Change — Number of Ways

```
PythonRun
def coin_change_count(coins: List[int], amount: int) -> int:
    dp = [0] * (amount + 1)
    dp[0] = 1

    for coin in coins:
        for total in range(coin, amount + 1):
            dp[total] += dp[total - coin]

    return dp[amount]
```

101. Longest Increasing Subsequence — $O(n^2)$

```
PythonRun
def lis_n2(nums: List[int]) -> int:
    if not nums:
        return 0

    dp = [1] * len(nums)

    for i in range(len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

102. Longest Increasing Subsequence — $O(n \log n)$

```
PythonRun
def lis_nlogn(nums: List[int]) -> int:
    tails = []

    for x in nums:
        index = bisect.bisect_left(tails, x)

        if index == len(tails):
            tails.append(x)
        else:
            tails[index] = x
```

```
return len(tails)
```

103. Longest Common Subsequence

PythonRun def longest_common_subsequence(text1: str, text2: str) -> int:

```
m = len(text1)
```

```
n = len(text2)
```

```
dp = [[0] * (n+1) for _ in range(m+1)]
```

```
for i in range(m-1, -1, -1):
```

```
    for j in range(n-1, -1, -1):
```

```
        if text1[i] == text2[j]:
```

```
            dp[i][j] = 1 + dp[i+1][j+1]
```

```
        else:
```

```
            dp[i][j] = max(dp[i+1][j], dp[i][j+1])
```

```
return dp[0][0]
```

104. Edit Distance

PythonRun def edit_distance(word1: str, word2: str) -> int:

```
m = len(word1)
```

```
n = len(word2)
```

```
dp = [[0] * (n+1) for _ in range(m+1)]
```

```
for i in range(m+1):
```

```
    dp[i][n] = m - i
```

```
for j in range(n+1):
```

```
    dp[m][j] = n - j
```

```
for i in range(m-1, -1, -1):
```

```
    for j in range(n-1, -1, -1):
```

```
        if word1[i] == word2[j]:
```

```
            dp[i][j] = dp[i+1][j+1]
```

```
        else:
```

```
            dp[i][j] = 1 + min(
```

```

dp[i+1][j],
dp[i][j+1],
dp[i+1][j+1]
    )

```

```

return dp[0][0]

```

105. Palindrome DP

PythonRun def longest_palindromic_substring(s: str) -> str:

```

n=len(s)

```

```

best=""

```

```

for center in range(n):

```

```

    for left, right in [(center, center), (center, center+1)]:

```

```

        while left >= 0 and right < n and s[left] == s[right]:

```

```

            if right-left+1 > len(best):

```

```

                best = s[left:right+1]

```

```

    left -= 1

```

```

    right += 1

```

```

return best

```

106. Interval DP

Used for matrix chain multiplication, burst balloons, merging stones.

PythonRun def interval_dp(nums: List[int]) -> int:

```

n=len(nums)

```

```

dp = [[0] * n for _ in range(n)]

```

```

for length in range(2, n+1):

```

```

    for left in range(n-length+1):

```

```

        right = left+length-1

```

```

        for mid in range(left, right):

```

```

            dp[left][right] = max(

```

```

                dp[left][right],

```

```

                dp[left][mid] + dp[mid+1][right]
            )

```

```
)  
  
returndp[0][n-1]
```

107. Bitmask DP

Used for travelling salesman, assignment, subsets.

```
PythonRundefbitmask_dp(cost: List[List[int]]) -> int:
```

```
n=len(cost)
```

```
full_mask= (1<<n) -1
```

```
@lru_cache(None)
```

```
defdp(i: int, mask: int) -> int:
```

```
    if i==n:
```

```
        return 0
```

```
    answer=float("inf")
```

```
    for j in range(n):
```

```
        if not (mask & (1<<j)):
```

```
            answer=min(
```

```
                answer,
```

```
                cost[i][j] + dp(i+1, mask | (1<<j))
```

```
            )
```

```
    return answer
```

```
return dp(0, 0)
```

108. Tree DP

```
PythonRundeftree_dp(root: Optional[TreeNode]) -> int:
```

```
defdfs(node: Optional[TreeNode]) -> Tuple[int, int]:
```

```
    if not node:
```

```
        return 0, 0
```

```
    left_take, left_skip=dfs(node.left)
```

```
    right_take, right_skip=dfs(node.right)
```

```
take=node.val +left_skip+right_skip
skip=max(left_take, left_skip) +max(right_take, right_skip)
```

```
returntake, skip
```

```
take, skip=dfs(root)
returnmax(take, skip)
```

109. Stock DP — State Machine

```
PythonRundefmax_profit_with_cooldown(prices: List[int]) -> int:
ifnotprices:
return0
```

```
hold=-prices[0]
sold=0
rest=0
```

```
forpriceinprices[1:]:
prev_hold=hold
prev_sold=sold
prev_rest=rest
```

```
hold=max(prev_hold, prev_rest-price)
sold=prev_hold+price
rest=max(prev_rest, prev_sold)
```

```
returnmax(sold, rest)
```

11. Greedy Patterns

110. Greedy Sort Then Choose

```
PythonRundefgreedy_sort_choose(nums: List[int]) -> int:
nums.sort()
```

```
answer=0
```

```
forxinnums:
```

```
if True:
    answer += x

return answer
```

111. Activity Selection / Non-Overlapping Intervals

PythonRun def erase_overlap_intervals(intervals: List[List[int]]) -> int:

```
intervals.sort(key=lambda x: x[1])
```

```
count = 0
prev_end = float("-inf")
```

```
for start, end in intervals:
    if start >= prev_end:
        count += 1
        prev_end = end
```

```
return len(intervals) - count
```

112. Jump Game

PythonRun def can_jump(nums: List[int]) -> bool:

```
farthest = 0
```

```
for i, jump in enumerate(nums):
    if i > farthest:
        return False
```

```
farthest = max(farthest, i + jump)
```

```
return True
```

113. Gas Station

PythonRun def can_complete_circuit(gas: List[int], cost: List[int]) -> int:

```
if sum(gas) < sum(cost):
    return -1
```

```
tank=0
start=0

for i in range(len(gas)):
    tank+=gas[i]-cost[i]

    if tank<0:
        start=i+1
        tank=0

return start
```

12. String Algorithm Patterns

114. Palindrome Two Pointers

```
PythonRundefis_palindrome(s: str) -> bool:
    left=0
    right=len(s)-1

    while left<right:
        while left<right and not s[left].isalnum():
            left+=1

        while left<right and not s[right].isalnum():
            right-=1

        if s[left].lower() != s[right].lower():
            return False

        left+=1
        right-=1

    return True
```

115. Anagram Check

```
PythonRundefis_anagram(s: str, t: str) -> bool:
    return Counter(s) == Counter(t)
```

116. KMP Pattern Matching

PythonRundefkmp_search(text: str, pattern: str) -> List[int]:

if not pattern:

return list(range(len(text) + 1))

lps = [0] * len(pattern)

length = 0

i = 1

while i < len(pattern):

if pattern[i] == pattern[length]:

length += 1

lps[i] = length

i += 1

elif length > 0:

length = lps[length - 1]

else:

lps[i] = 0

i += 1

result = []

i = 0

j = 0

while i < len(text):

if text[i] == pattern[j]:

i += 1

j += 1

if j == len(pattern):

result.append(i - j)

j = lps[j - 1]

elif j > 0:

j = lps[j - 1]

else:

i += 1

return result

117. Rolling Hash / Rabin-Karp

PythonRunde
def rabin_karp(text: str, pattern: str) -> List[int]:

if len(pattern) > len(text):

return []

base = 256

mod = 10**9+7

m = len(pattern)

pattern_hash = 0

window_hash = 0

highest_power = 1

for i in range(m):

pattern_hash = (pattern_hash * base + ord(pattern[i])) % mod

window_hash = (window_hash * base + ord(text[i])) % mod

if i < m - 1:

highest_power = (highest_power * base) % mod

result = []

for i in range(len(text) - m + 1):

if pattern_hash == window_hash:

if text[i:i+m] == pattern:

result.append(i)

if i < len(text) - m:

window_hash = (

(window_hash - ord(text[i]) * highest_power) * base

+ ord(text[i+m])

) % mod

return result

118. Z Algorithm

```
PythonRundefz_algorithm(s: str) -> List[int]:
n=len(s)
z= [0] *n
left=0
right=0

foriinrange(1, n):
ifi<=right:
z[i] =min(right-i+1, z[i-left])

whilei+z[i] <nands[z[i]] ==s[i+z[i]]:
z[i] +=1

ifi+z[i] -1>right:
left=i
right=i+z[i] -1

returnz
```

13. Matrix Patterns

119. Direction Array

```
PythonRundefmatrix_traversal(grid: List[List[int]]) -> None:
rows=len(grid)
cols=len(grid[0])

directions= [(1, 0), (-1, 0), (0, 1), (0, -1)]

forrinrange(rows):
forcinrange(cols):
fordr, dcindirections:
nr=r+dr
nc=c+dc

if0<=nr<rowsand0<=nc<cols:
pass
```

120. Spiral Matrix

```
PythonRundefspiral_order(matrix: List[List[int]]) -> List[int]:  
result= []
```

```
top=0  
bottom=len(matrix) -1  
left=0  
right=len(matrix[0]) -1  
  
whiletop<=bottomandleft<=right:  
    forcinrange(left, right+1):  
        result.append(matrix[top][c])  
        top+=1
```

```
    forrinrange(top, bottom+1):  
        result.append(matrix[r][right])  
        right-=1
```

```
    iftop<=bottom:  
        forcinrange(right, left-1, -1):  
            result.append(matrix[bottom][c])  
            bottom-=1
```

```
    ifleft<=right:  
        forrinrange(bottom, top-1, -1):  
            result.append(matrix[r][left])  
            left+=1
```

```
returnresult
```

121. 2D Prefix Sum

```
PythonRunclassNumMatrix:  
def__init__(self, matrix: List[List[int]]):  
    rows=len(matrix)  
    cols=len(matrix[0])  
  
    self.prefix = [[0] * (cols+1) for_inrange(rows+1)]  
  
    forrinrange(rows):
```

```

for c in range(cols):
    self.prefix[r+1][c+1] = (
        matrix[r][c]
        +self.prefix[r][c+1]
        +self.prefix[r+1][c]
        -self.prefix[r][c]
    )

def sum_region(self, r1: int, c1: int, r2: int, c2: int) -> int:
    return (
        self.prefix[r2+1][c2+1]
        -self.prefix[r1][c2+1]
        -self.prefix[r2+1][c1]
        +self.prefix[r1][c1]
    )

```

14. Advanced Data Structure Patterns

122. Fenwick Tree / Binary Indexed Tree

Python Run class FenwickTree:

```

def __init__(self, n: int):
    self.n = n
    self.tree = [0] * (n+1)

```

```

def update(self, index: int, delta: int) -> None:
    index += 1

```

```

while index <= self.n:
    self.tree[index] += delta
    index += index & -index

```

```

def query(self, index: int) -> int:
    index += 1
    result = 0

```

```

while index > 0:
    result += self.tree[index]
    index -= index & -index

```

```
returnresult
```

```
defrange_query(self, left: int, right: int) -> int:  
returnself.query(right) -self.query(left-1)
```

123. Segment Tree

```
PythonRunclassSegmentTree:
```

```
def__init__(self, nums: List[int]):
```

```
self.n =len(nums)
```

```
self.tree = [0] * (4*self.n)
```

```
self.build(nums, 0, 0, self.n -1)
```

```
defbuild(self, nums: List[int], node: int, left: int, right: int) -> None:
```

```
ifleft==right:
```

```
self.tree[node] =nums[left]
```

```
return
```

```
mid=left+ (right-left) //2
```

```
self.build(nums, 2*node+1, left, mid)
```

```
self.build(nums, 2*node+2, mid+1, right)
```

```
self.tree[node] =self.tree[2*node+1] +self.tree[2*node+2]
```

```
defupdate(self, index: int, value: int, node: int=0, left: int=0, right: Optional[int] =None) ->  
None:
```

```
ifrightisNone:
```

```
right=self.n -1
```

```
ifleft==right:
```

```
self.tree[node] =value
```

```
return
```

```
mid=left+ (right-left) //2
```

```
ifindex<=mid:
```

```
self.update(index, value, 2*node+1, left, mid)
```

```
else:
```

```
self.update(index, value, 2*node+2, mid+1, right)
```

```

self.tree[node] =self.tree[2*node+1] +self.tree[2*node+2]

defquery(self, ql: int, qr: int, node: int=0, left: int=0, right: Optional[int] =None) -> int:
    ifrightisNone:
        right=self.n -1

    ifqr<leftorright<ql:
        return0

    ifql<=leftandright<=qr:
        returnself.tree[node]

    mid=left+ (right-left) //2

    return (
        self.query(ql, qr, 2*node+1, left, mid)
        +self.query(ql, qr, 2*node+2, mid+1, right)
    )

```

15. Bit Manipulation Patterns

124. Check, Set, Clear, Toggle Bit

```

PythonRundefbit_operations(x: int, i: int) -> Tuple[bool, int, int, int]:
    is_set= (x& (1<<i)) !=0
    set_bit=x| (1<<i)
    clear_bit=x&~(1<<i)
    toggle_bit=x^ (1<<i)

    returnis_set, set_bit, clear_bit, toggle_bit

```

125. Count Set Bits

```

PythonRundefcount_bits(x: int) -> int:
    count=0

    whilex:
        x&=x-1
        count+=1

```

```
returncount
```

126. XOR Single Number

```
PythonRundefsingle_number(nums: List[int]) -> int:  
result=0
```

```
forxinnums:  
result^=x
```

```
returnresult
```

127. Enumerate All Submasks

```
PythonRundefenumerate_submasks(mask: int) -> List[int]:  
result= []  
submask=mask
```

```
whilesubmask:  
result.append(submask)  
submask= (submask-1) &mask
```

```
result.append(0)  
returnresult
```

128. Generate All Subsets Using Bitmask

```
PythonRundefsubsets_bitmask(nums: List[int]) -> List[List[int]]:  
n=len(nums)  
result= []
```

```
formaskinrange(1<<n):  
subset= []
```

```
foriinrange(n):  
ifmask& (1<<i):  
subset.append(nums[i])
```

```
result.append(subset)
```

```
return result
```

16. Math and Number Theory Patterns

129. GCD and LCM

```
PythonRundefgcd_lcm(a: int, b: int) -> Tuple[int, int]:
```

```
g=math.gcd(a, b)
```

```
l=abs(a*b) //g
```

```
return g, l
```

130. Fast Power

```
PythonRundeffast_power(x: int, n: int) -> int:
```

```
result=1
```

```
while n>0:
```

```
if n&1:
```

```
result*=x
```

```
x*=x
```

```
n>>=1
```

```
return result
```

131. Modular Exponentiation

```
PythonRundefmod_power(x: int, n: int, mod: int) -> int:
```

```
result=1
```

```
x%=mod
```

```
while n>0:
```

```
if n&1:
```

```
result= (result*x) %mod
```

```
x= (x*x) %mod  
n>=1
```

```
returnresult
```

132. Sieve of Eratosthenes

```
PythonRundefsieve(n: int) -> List[int]:
```

```
is_prime= [True] * (n+1)
```

```
is_prime[0] =False
```

```
is_prime[1] =False
```

```
forpinrange(2, int(n**0.5) +1):
```

```
    ifis_prime[p]:
```

```
        formultipleinrange(p*p, n+1, p):
```

```
            is_prime[multiple] =False
```

```
return [iforiinrange(n+1) ifis_prime[i]]
```

133. Prime Factorization

```
PythonRundefprime_factors(n: int) -> List[int]:
```

```
factors= []
```

```
d=2
```

```
whiled*d<=n:
```

```
    whilen%d==0:
```

```
        factors.append(d)
```

```
        n//=d
```

```
d+=1
```

```
ifn>1:
```

```
    factors.append(n)
```

```
returnfactors
```

134. Combinations nCr

```
PythonRundefncr(n: int, r: int) -> int:
    if r < 0 or r > n:
        return 0

    r = min(r, n-r)
    result = 1

    for i in range(1, r+1):
        result = result * (n-r+i) // i

    return result
```

17. Design Pattern Questions

135. LRU Cache

```
PythonRunclassLRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1

        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)

        self.cache[key] = value

        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
```

136. Randomized Set

PythonRunimportrandom

```
classRandomizedSet:
    def__init__(self):
        self.values = []
        self.index = {}

    definsert(self, val: int) -> bool:
        ifvalinself.index:
            returnFalse

        self.index[val] =len(self.values)
        self.values.append(val)

    returnTrue

    defremove(self, val: int) -> bool:
        ifvalnotinself.index:
            returnFalse

        remove_index=self.index[val]
        last_value=self.values[-1]

        self.values[remove_index] =last_value
        self.index[last_value] =remove_index

        self.values.pop()
        delself.index[val]

    returnTrue

    defget_random(self) -> int:
        returnrandom.choice(self.values)
```

137. Min Stack

PythonRunclassMinStack:
 def__init__(self):
 self.stack = []

```

self.min_stack = []

def push(self, val: int) -> None:
    self.stack.append(val)

    if not self.min_stack:
        self.min_stack.append(val)
    else:
        self.min_stack.append(min(val, self.min_stack[-1]))

def pop(self) -> None:
    self.stack.pop()
    self.min_stack.pop()

def top(self) -> int:
    return self.stack[-1]

def get_min(self) -> int:
    return self.min_stack[-1]

```

138. Queue Using Two Stacks

```

Python Run class MyQueue:
    def __init__(self):
        self.input_stack = []
        self.output_stack = []

    def push(self, x: int) -> None:
        self.input_stack.append(x)

    def pop(self) -> int:
        self.peak()
        return self.output_stack.pop()

    def peek(self) -> int:
        if not self.output_stack:
            while self.input_stack:
                self.output_stack.append(self.input_stack.pop())

        return self.output_stack[-1]

```

```
defempty(self) -> bool:
returnnotself.input_stack andnotself.output_stack
```

139. Stack Using Queues

```
PythonRunclassMyStack:
```

```
def__init__(self):
self.queue =deque()
```

```
defpush(self, x: int) -> None:
self.queue.append(x)
```

```
for_inrange(len(self.queue) -1):
self.queue.append(self.queue.popleft())
```

```
defpop(self) -> int:
returnself.queue.popleft()
```

```
deftop(self) -> int:
returnself.queue[0]
```

```
defempty(self) -> bool:
returnnotself.queue
```

18. Reservoir Sampling

140. Reservoir Sampling

Used when stream size is unknown.

```
PythonRundefreservoir_sampling(stream: List[int], k: int) -> List[int]:
reservoir= []
```

```
fori, valueinenumerate(stream):
ifi<k:
reservoir.append(value)
else:
j=random.randint(0, i)
```

```
if j < k:
    reservoir[j] = value

return reservoir
```

19. Cyclic Sort Pattern

Used when array values are in range 1 to n.

```
PythonRun def cyclic_sort(nums: List[int]) -> None:
    i = 0
```

```
    while i < len(nums):
        correct_index = nums[i] - 1
```

```
        if nums[i] != nums[correct_index]:
            nums[i], nums[correct_index] = nums[correct_index], nums[i]
        else:
            i += 1
```

```
PythonRun def find_missing_number(nums: List[int]) -> int:
    i = 0
    n = len(nums)
```

```
    while i < n:
        correct_index = nums[i]
```

```
        if nums[i] < n and nums[i] != nums[correct_index]:
            nums[i], nums[correct_index] = nums[correct_index], nums[i]
        else:
            i += 1
```

```
    for i in range(n):
        if nums[i] != i:
            return i
```

```
    return n
```

20. Complete Pattern Checklist

Use this checklist while solving coding problems:

Category	Patterns
Arrays	single pass, two pointers, sliding window, prefix sum, difference array, Kadane, cyclic sort
Strings	frequency map, sliding window, palindrome, KMP, rolling hash, trie
Linked List	dummy node, fast/slow, reverse, merge, cycle detection, k-group reverse
Stack	parentheses, monotonic stack, histogram, min stack
Queue	BFS, level order, monotonic queue, queue using stacks
Binary Search	normal search, bounds, rotated array, binary search on answer
Heap	top K, k-way merge, two heaps, scheduling, lazy deletion
Trees	DFS, BFS, BST, LCA, serialization, tree DP
Graphs	BFS, DFS, topo sort, union find, Dijkstra, Bellman-Ford, MST, bipartite
Backtracking	subsets, permutations, combinations, partitioning, board search, N-Queens
DP	memoization, bottom-up, 1D, 2D, knapsack, LIS, LCS, interval DP, bitmask DP
Greedy	sorting, intervals, jump game, gas station

Category	Patterns
Bit Manipulation	masks, XOR, bit count, submask enumeration
Math	gcd, lcm, sieve, fast power, combinatorics
Design	LRU, randomized set, min stack, queue/stack conversion

The big trick is: in interviews, the problem statement changes costume, but the pattern underneath is usually one of these. Same villain, different hat.

Branched from [Coding Interview Patterns](#)